

EMPLOYEE RECORD STORAGE USING HASH TABLES

Efficient Employee Record Management Using Hashing and Separate Chaining

PROBLEM STATEMENT

- Managing a large number of employee records requires efficient storage and retrieval.
- Sequential searching can become time-consuming as the number of records increases.
- Employee management requires frequent insertion, searching, updating, and deletion operations.
- A hash table can provide fast access by using the Employee ID as a unique key.
- Therefore, a hash-table-based system is developed for efficient employee record management.

	A25126510025
	A25126510018
	A25126510020
	A25126510033
	A25126510055

OBJECTIVES

- To store employee records using a hash table.
- To use Employee ID as a unique key for each record.
- To perform insertion, searching, updating, and deletion efficiently.
- To handle hash collisions using separate chaining.
- To maintain performance through load-factor monitoring and rehashing.

PROJECT OVERVIEW

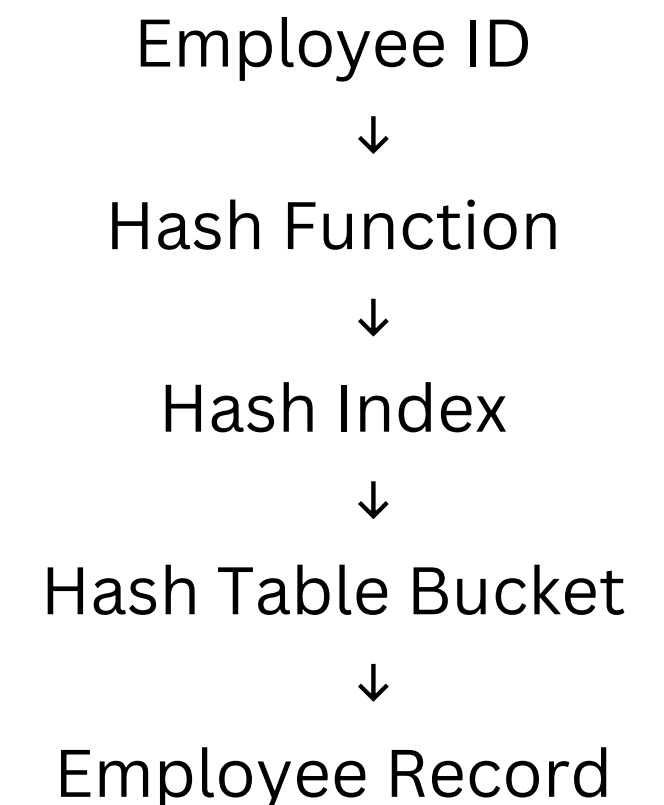
- The system is implemented using Java.
- Employee records are stored inside a hash table.
- Employee ID is used to calculate the hash-table index.
- Separate chaining is used when multiple records map to the same index.
- The system provides a menu-driven interface for record management.

Employee Record Contains

- Employee ID
- Name
- Age
- Qualification
- Position
- Salary

HASH TABLE

- A hash table is a data structure used to store records using a key.
- It uses a hash function to convert a key into a table index.
- In this project, Employee ID is the key.
- The calculated index determines where the employee record is stored.
- This provides efficient access to employee records.



HASH FUNCTION

Working Principle

- The Employee ID is given as input to the hash function.
- The table size is used to calculate the bucket index.
- The calculated index determines the storage location.
- The same calculation is used during searching.
- This allows the system to directly locate the appropriate bucket.

$$\text{Index} = \text{Employee ID} \% \text{Table Size}$$

SEPARATE CHAINING

- Separate chaining stores collided records in a linked list.
- Each hash-table bucket acts as the head of a chain.
- A new collided employee is added to the existing chain.
- Searching traverses the chain until the required Employee ID is found.
- This allows multiple records to share the same hash index.

HASH COLLISION

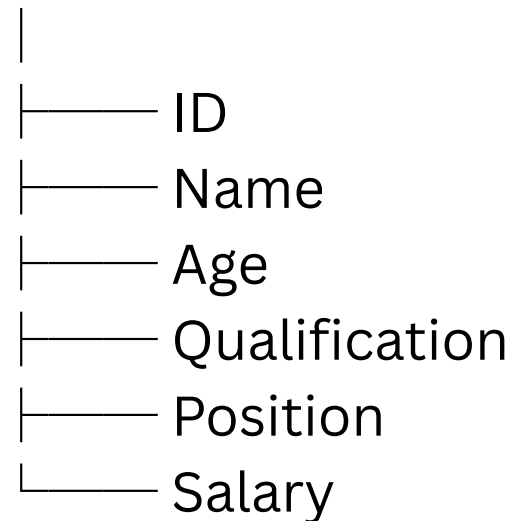
Working Principle

- A collision occurs when two different Employee IDs produce the same hash index.
- Multiple records cannot simply replace one another.
- Therefore, a collision-resolution technique is required.
- This project uses separate chaining.
- Each bucket can contain a linked chain of employee records.



EMPLOYEE RECORD

Employee



Employee ID uniquely identifies the employee.

Name stores the employee's name.

Age stores the employee's age.

Qualification stores educational qualification.

Position and Salary store employment details.

- Employee — stores employee details
- Node — linked-list node
- EmployeeHashTable — hash table operations
- Input — input validation
- EmployeeRecordSystem — main menu & operations

STEP-BY-STEP LOGIC

Step 1: Create the employee record.

Step 2: Read the Employee ID and employee details.

Step 3: Check whether the Employee ID already exists.

Step 4: Calculate the hash index.

Step 5: Insert the record into the corresponding bucket.

Step 6: If the bucket already contains a record, use separate chaining.

Step 7: Perform search, update, or deletion using the Employee ID.

OPERATIONS

Insert Employee

Search Employee

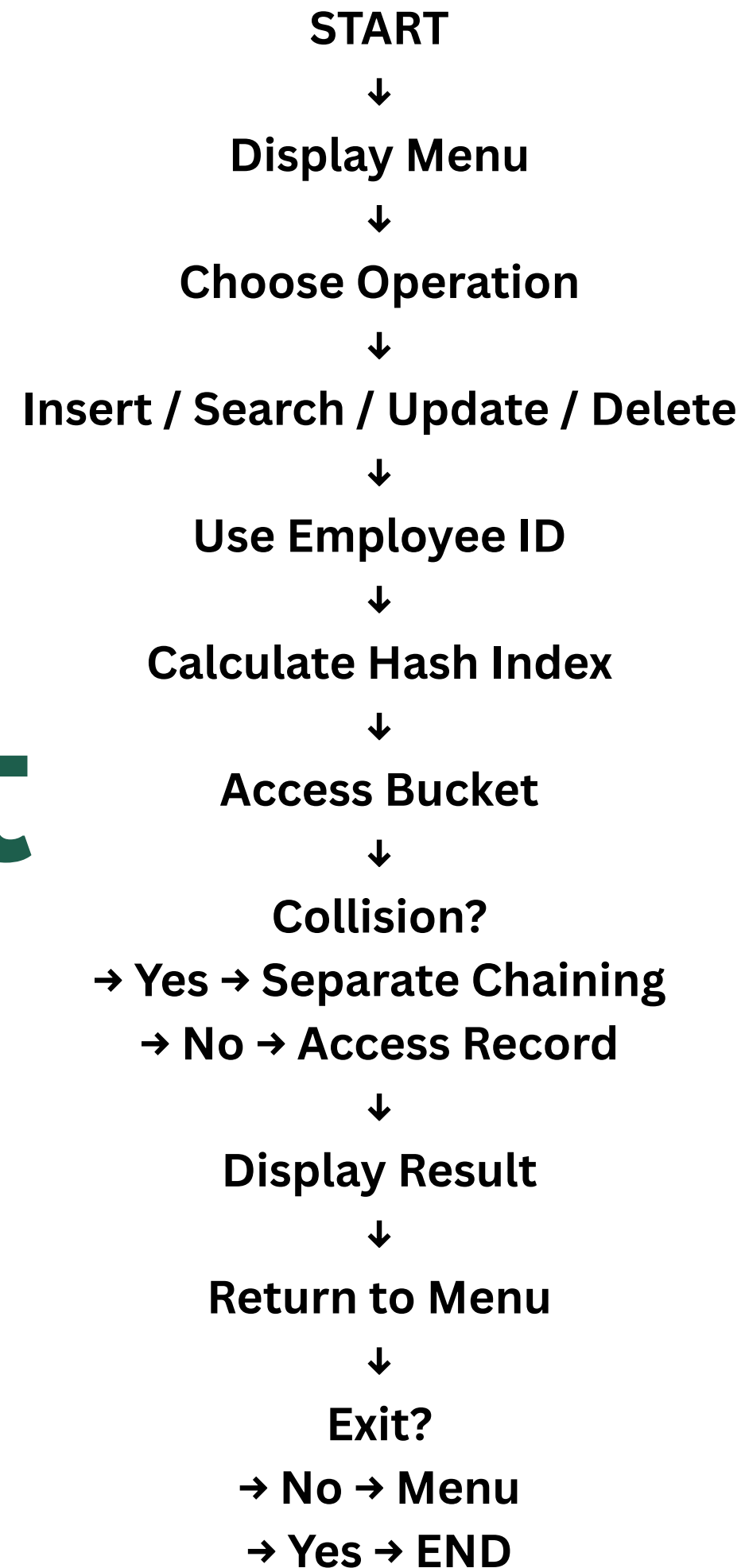
Update Employee

Delete Employee

Display All Employees

View Hash Statistics

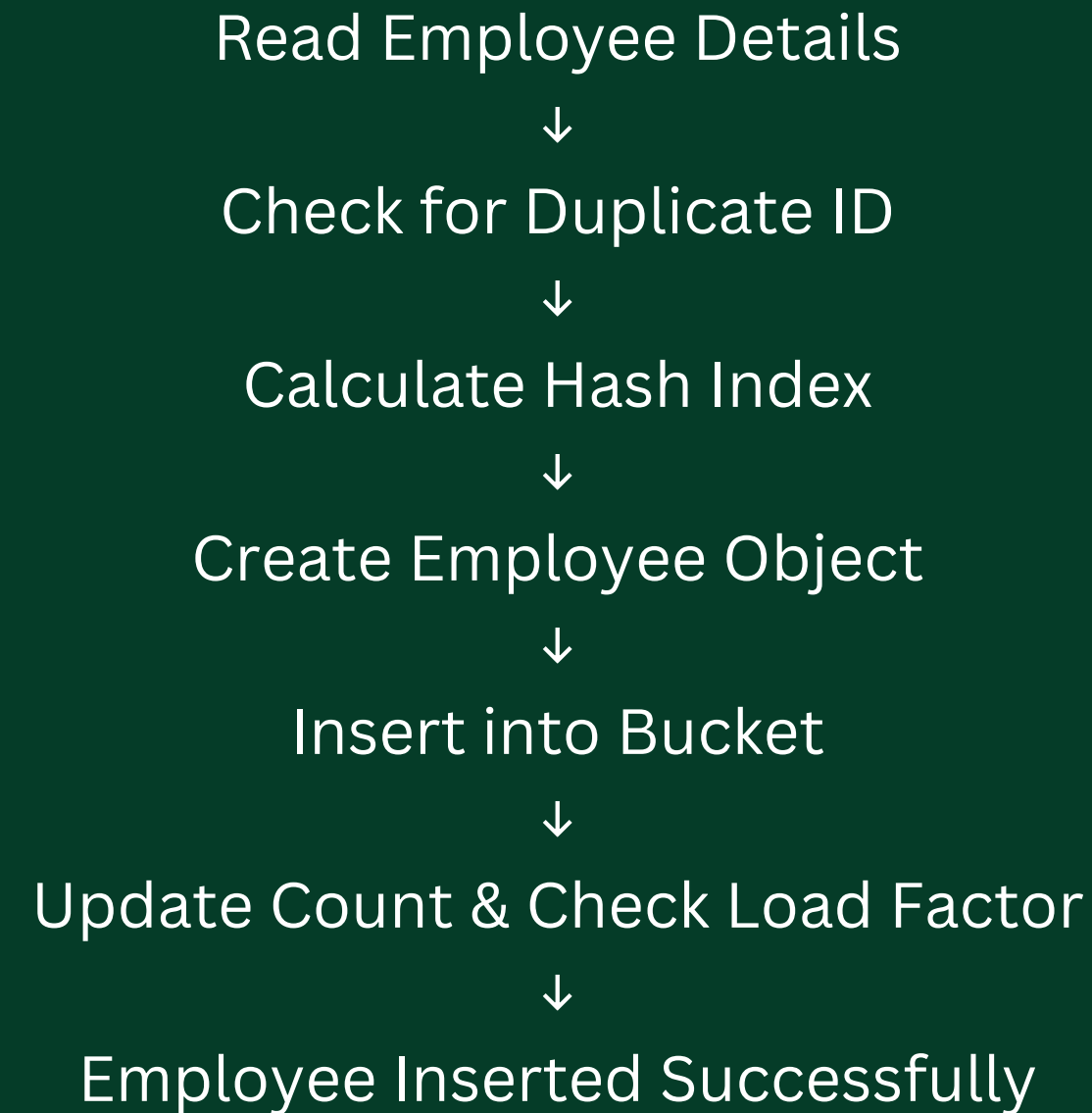
Flow chart



Choice Function

- 1.insertEmployee()
- 2.searchEmployee()
- 3.updateEmployee()
- 4.deleteEmployee()
- 5.displayAll()
- 6.displayStatistics()
- 7.Exit

1. INSERT EMPLOYEE



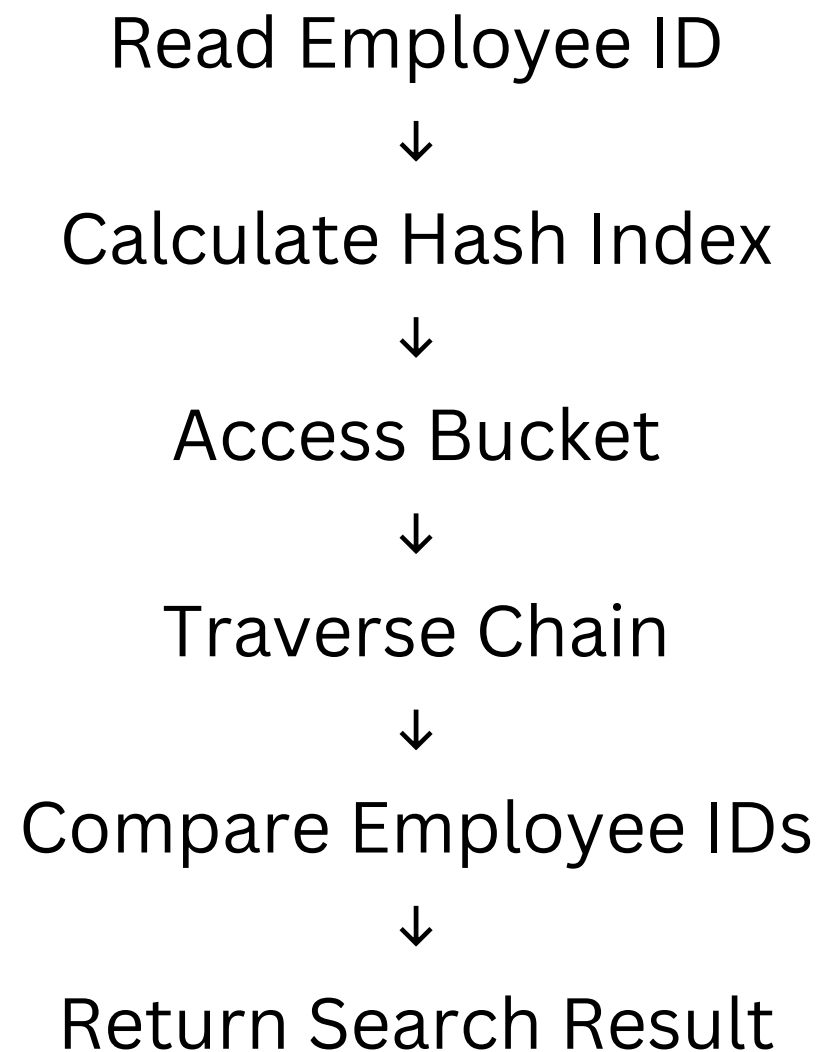
Employee ID = 101

Table Size = 5

Hash calculation: $101 \% 5 = 1$

Employee is stored in Bucket 1

2. SEARCH EMPLOYEE



Search ID = 101

Hash index: $101 \% 5 = 1$

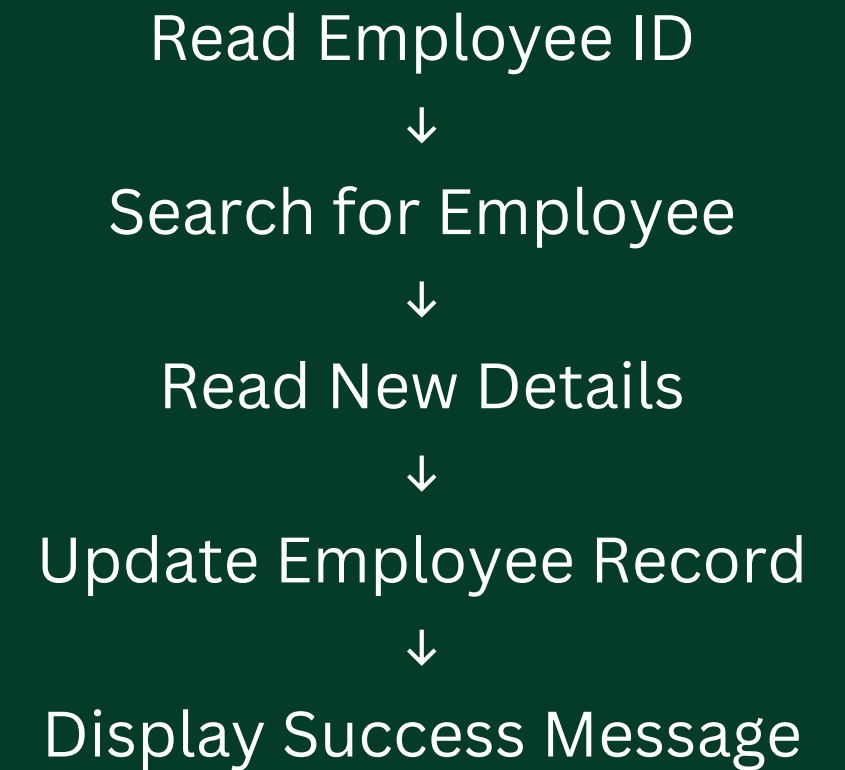
Access Bucket 1

Chain: 101 → 106 → 111

ID 101 found

Employee record returned

3. UPDATE EMPLOYEE



Before Update — ID 101

Name: Rahul

Age: 25

Position: Developer

Salary: ₹50,000

↓ Update

After Update — ID 101

Name: Rohan

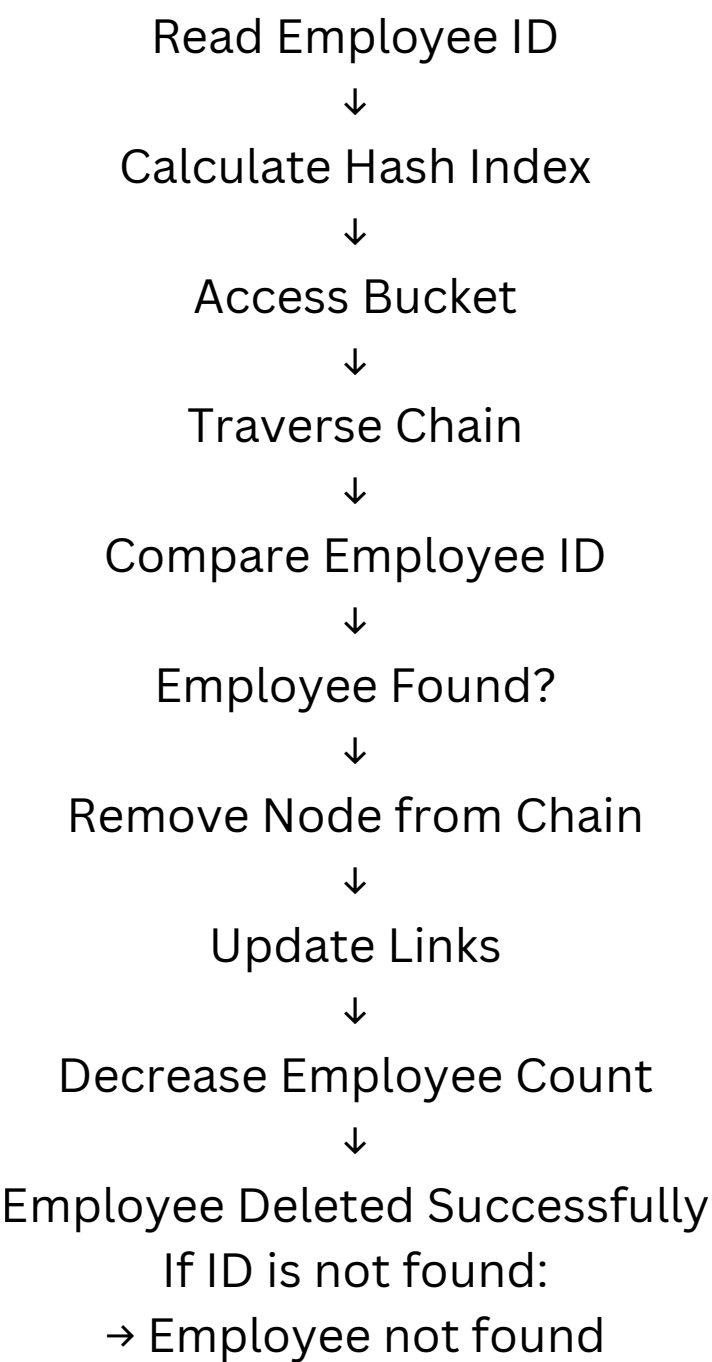
Age: 26

Position: Senior Developer

Salary: ₹70,000

Result: Employee record successfully updated.

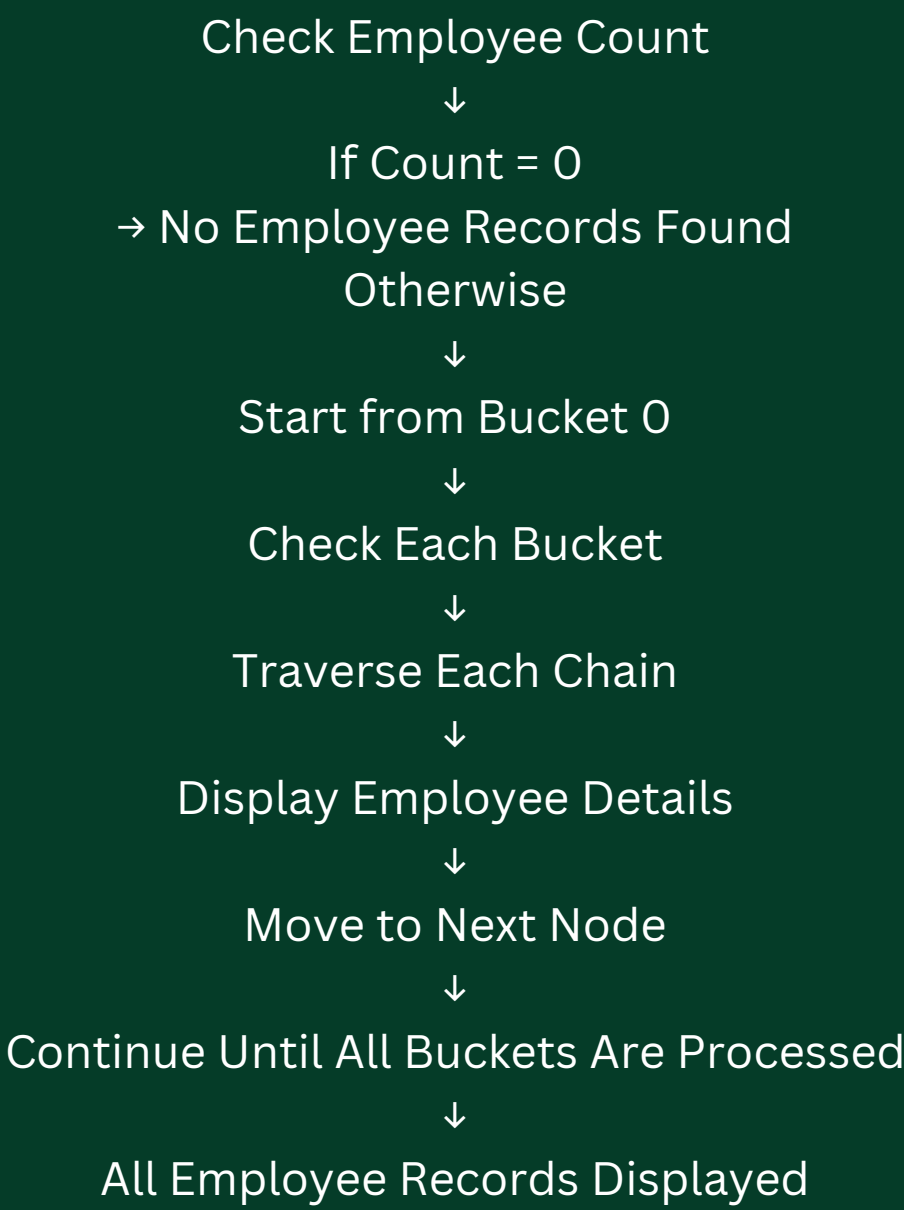
4. DELETE EMPLOYEE



Employee ID = 101
Hash Index = 101 % 5 = 1
Bucket 1 contains: 101 → 106 → 111
Search for ID 101
Employee found
Remove 101 from the chain

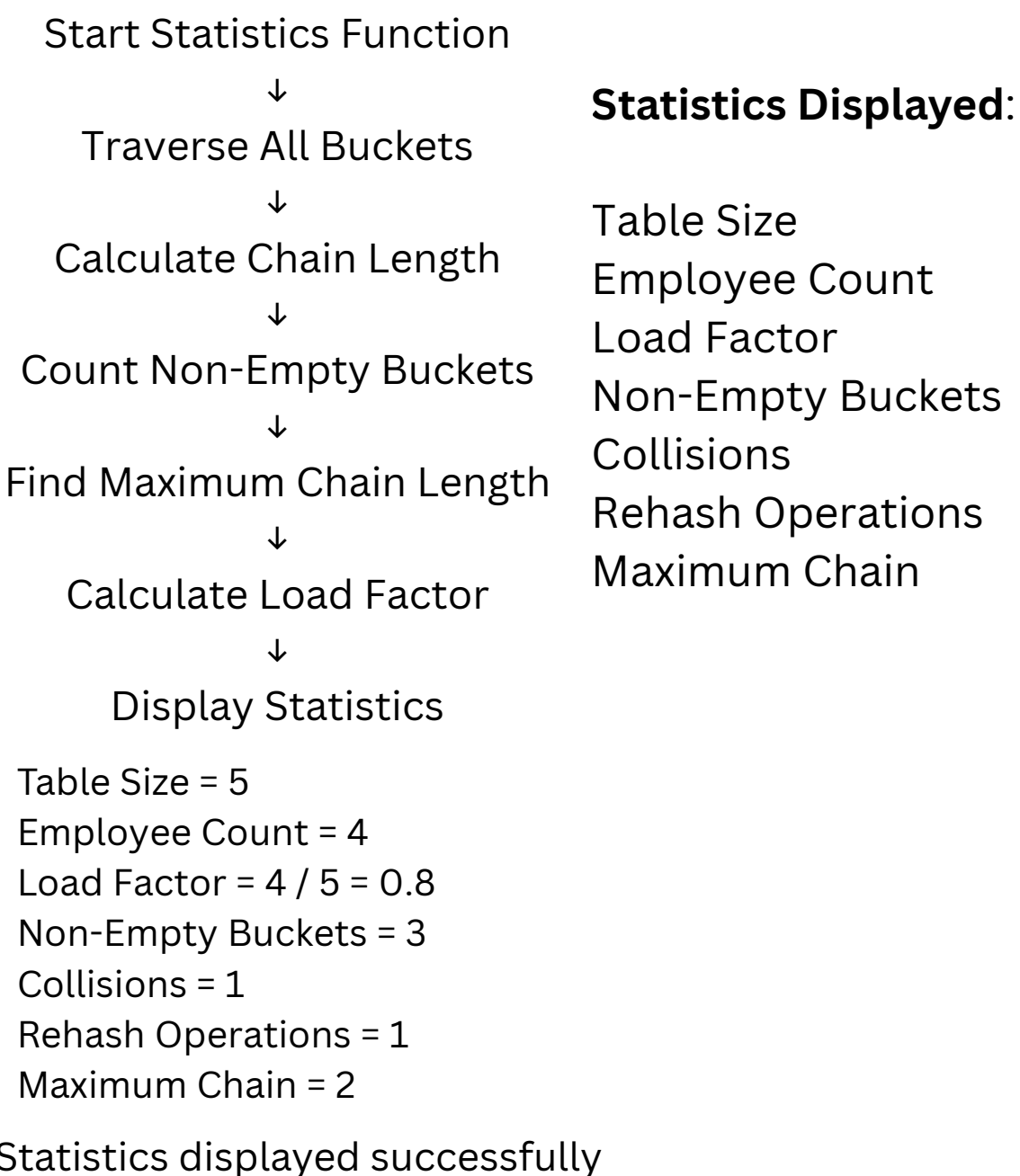
After Deletion:
106 → 111
Employee deleted successfully

5. DISPLAY ALL EMPLOYEES

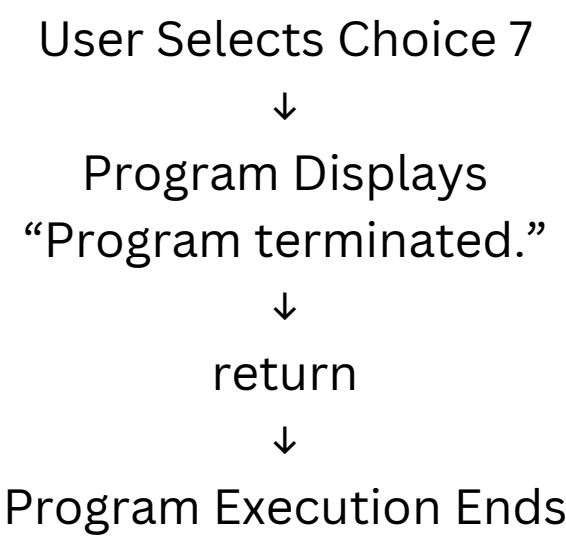


Hash Table:
Bucket 0 → Empty
Bucket 1 → 101 → 106
Bucket 2 → 102
Bucket 3 → Empty
Bucket 4 → 104
Function traverses all buckets and chains
Output:
Employee 101
Employee 106
Employee 102
Employee 104
All employee records displayed

6. HASH TABLE STATISTICS



7. EXIT



Collision Handling

Collision occurs when two Employee IDs produce the same hash index.

Table Size = 5

ID 101 $\rightarrow 101 \% 5 = 1$

ID 106 $\rightarrow 106 \% 5 = 1$

ID 111 $\rightarrow 111 \% 5 = 1$

Bucket 1:

101 $\rightarrow$ 106 $\rightarrow$ 111

Technique Used: Separate Chaining

Records are connected using linked nodes.

New collided records are added to the chain.

Rehashing

Rehashing means making the hash table bigger when it becomes too full.

Table Size = 5

Employees = 4

Load Factor = $4/5 = 0.80$

$0.80 > 0.75 \rightarrow$ Rehashing

Create a larger table

Move employees to new hash positions.

Table becomes full



Check Load Factor



If Load Factor >

0.75



Create a bigger
table



Move existing
employees



Calculate new
positions



Continue using the
new table



Complexity Analysis

 Operation	 Best Case	 Average Case	 Worst Case
 Insert	$O(1)$	$O(1)$	$O(n)$
 Search	$O(1)$	$O(1)$	$O(n)$
 Update	$O(1)$	$O(1)$	$O(n)$
 Delete	$O(1)$	$O(1)$	$O(n)$

Reason:

Operations use the Employee ID to directly locate the hash bucket. In the worst case, records may form a long chain.

Advantages

- Fast employee record access
- Efficient Insert, Search, Update and Delete
- Employee ID used as unique key
- Handles collisions using separate chaining
- Automatic rehashing
- Menu-driven and easy to use
- Maintains hash table statistics

Limitations

- Performance depends on hash distribution.
- Long chains can increase search time.
- Memory is required for linked nodes.
- Rehashing requires moving existing records.
- Data is stored only during program execution.

Future Enhancements

- Database integration
- Graphical User Interface (GUI)
- File-based permanent storage
- Advanced employee filtering
- Authentication and user access control
- Export employee records to CSV/PDF

Conclusion

- Employee Record Storage Using Hash Tables
- Provides efficient employee record management.
- Uses Employee ID as the hash key.
- Supports Insert, Search, Update and Delete.
- Handles collisions using Separate Chaining.
- Uses Rehashing to maintain performance.
- Demonstrates practical use of Hash Tables and Linked Lists.